

Software Development Fundamentals (SDF) – I

ODD 2023



Pointers – Pointer Arithmetic, Pointer to 2D arrays, Dynamic Memory Allocation

Jaypee Institute of Information Technology (JIIT), Noida

Updated by Amitesh, Dr. Ashish Mishra on 14-09-2023. If any errors or correction is required – kindly report it on Amitesh@mail.jiit.ac.in; Ashish.Mishra@mail.jiit.ac.in

Outline

- Arithmetic operation on Pointers
- Pointers and 2D Arrays / Multi-Dimensional Arrays
- Pointers to arrays and dynamic memory allocation

Arithmetic Operation on Pointers

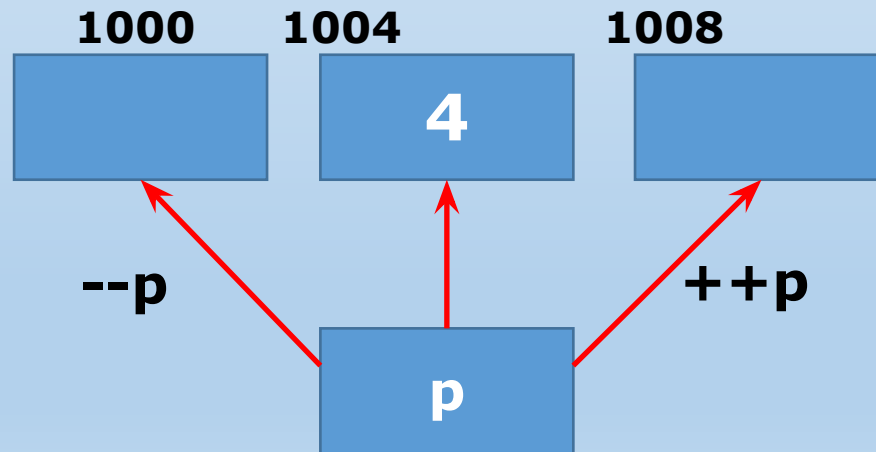
- **Pointers refer to address of a variable and are also referred as “address data types.”**
- **C does not allow all arithmetic operations on pointers.**
- **Possible arithmetic operations on pointers are:**
 - **Increment/Decrement of a Pointer.**
 - **Addition/Subtraction of an integer to a pointer**
 - **Subtracting two pointers of same type**
- **Addition of two or more pointers is not allowed.**
- **Pointers can not be multiplied/divided with an integer or other pointer.**

Increment or Decrement of Pointers

- Increment or decrement of a pointer means pointing to next location or previous location.

For example, let us assume we have a integer pointer and we know that size of integer is 4.

```
int a=4,*p;  
p=&a;
```



Operations are meaningful when used with pointers to arrays

Example

```
#include<stdio.h>
int main()
{
    int number=50;
    int *p;          //pointer to int
    p=&number;        //stores the address of number variable
    printf("Address of p variable is %p \n",p);
    p=p+1;
    printf("After increment: Address of p variable is %p \n",p);
    return 0;
}
```

Output: (addresses may vary)

Address of p variable is 0x7ffec09c97d4 After increment: Address of p variable is 0x7ffec09c97d8

Example with array

```
#include<stdio.h>
int main()
{
    int arr[5] = {1, 2, 3, 4, 5};
    int *p = arr;
    int i;
    printf("printing array elements...\n");
    for(i = 0; i < 5; i++)
    {
        printf("%d \n ",*(p++));
    }
}
```

Output:

1
2
3
4
5

Addition/Subtraction of an integer to a pointer

```
int *P;
```

Assuming that P is pointing to address 1000 and sizeof(int)=4

Expression	Meaning
$P = P + 1$	$P = P + 1 = 1000 + 4 = 1004$
$++P$ or $P++$	$P = P + 1 = 1000 + 4 = 1004$
$P = P + 5$	$P = P + 5 = 1000 + 5 * 4 = 1020$
$P = P - 2$	$P = P - 2 = 1000 + 2 * 4 = 992$
$P--$ or $--P$	$P = P - 1 = 1000 - 4 = 996$

We can get similar results for pointer pointing to any data type by replacing the **size of int with **size of other data type**.**

Addition/Subtraction of an integer to a pointer

```
#include<stdio.h>
int main()
{
    int number=50;
    int *p;          //pointer to int
    p=&number;        //stores the address of number variable
    printf("Address of p variable is %p \n",p);
    p=p-3;           //subtracting 3 from pointer variable
    printf("After subtracting 3: Address of p variable is %p \n",p);
    return 0;
}
```

Output: (addresses may vary)

Address of p variable is 0x7ffd16eea594

After subtracting 3: Address of p variable is 0x7ffd16eea588

Subtracting two pointers of same type

```
#include<stdio.h>
void main ()
{
    int i = 100;
    int *p = &i;
    int *q;
    q = p;
    p = p + 3;
    printf("Pointer Subtraction: %p - %p = %p",p, q, p-q);
}
```

Output: (addresses may vary)

Pointer Subtraction:0x7ffc22e5f828-0x7ffc22e5f81c=0x3

End of Lecture

Outline

- **Pointers and 2D Arrays / Multi-Dimensional Arrays**

Pointers to 2D Array

- A 2-D array is considered as an array of arrays, for example each row of a 2-D array can be considered as a 1-D array.

```
int arr[2][5] = {{1,2,3,4,5},{6,7,8,9,10}};
```

1	2	3	4	5	6	7	8	9	10
1000	1004	1008	1012	1016	1020	1024	1028	1032	1036
arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]	arr[0][4]	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]	arr[1][4]

The array is stored in row-major order.

Pointer Declaration

- Declaring a pointer to 2-D array
 - **<data-type of array> <pointer_variable>[column size] = array-name;**

- For example:

The **(*prr)** declares a pointer to an array.

int (*parr)[5] = arr;

[5] will inform that prr is a pointer to how many integers, as a 2-D array is array of arrays so it defines element size of 1 row.

Base address of the array

Access Array Element

1	2	3	4	5	6	7	8	9	10
1000	1004	1008	1012	1016	1020	1024	1028	1032	1036
arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]	arr[0][4]	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]	arr[1][4]

- **prr => 1000**
- **prr + 1 => 1020 i.e., *prr + size of 1 row**
- **(prr + i) => gives the starting address of ith row.**
- ***(prr + i) + j => points to address of jth element of ith row**
- ***(*(prr+i)+j) => gives the value of jth element of ith row**

Example

main.c



Run

Output

```
1  #include<stdio.h>
2  int main()
3  {
4      int arr[3][4] = {{11,22,33,44}, {55,66,77,88}, {11,66,77,44}}
5          ;
6      int i, j;
7      int (*p)[4];
8      p = arr;
9      for(i = 0; i < 3; i++)
10     {
11         printf("Address of array at row %d: %p \n",i , p + i);
12         for(j = 0; j < 4; j++)
13         {
14             printf("arr[%d][%d]=%d\n", i, j, *( *(p + i) + j) );
15         }
16         printf("\n\n");
17     }
18     return 0;
19 }
```

/tmp/3vbLEhz4z1.o

Address of array at row 0: 0x7ffea49cd560

arr[0][0]=11

arr[0][1]=22

arr[0][2]=33

arr[0][3]=44

Address of array at row 1: 0x7ffea49cd570

arr[1][0]=55

arr[1][1]=66

arr[1][2]=77

arr[1][3]=88

Address of array at row 2: 0x7ffea49cd580

arr[2][0]=11

arr[2][1]=66

arr[2][2]=77

arr[2][3]=44

Multidimensional Arithmetic

```
int arr[3][4][5];
```

Expression	Meaning
arr	An array of 3 2-D arrays.
arr+2	3 rd element of the array (in 1 st 2-D array)
*(arr+2)	Accessing 2-D array (3 rd 2-D array)
*(arr+2) + 1	Address of 2 nd 1-D array of 3 rd 2-D array
((arr+2)+1)	Accessing 2 nd 1-D array of 3 rd 2-D array
((arr+2)+1) + 2	Address of 3 rd element of 2 nd 1-D array of 3 rd 2-D array
((*(arr+2)+1)+2)	Accessing 3 rd element of 2 nd 1-D array of 3 rd 2-D array

Similar approach can be used to access element or address of array of any dimension

End of Lecture

Outline

- **Pointers to arrays and dynamic memory allocation**

Problem with Array

- **Sometimes**
 - Amount of data cannot be predicted beforehand, may change during program execution
- **Example: Search for an element in an array of N elements**
- **One solution: find the maximum possible value of N and allocate an array of N elements**
 - **Memory Wastage**
 - **Example: maximum value of N may be 10,000, but a particular run may need to search only among 100 elements**
 - **Using array of size 10,000 always wastes memory in most cases**

Solution

- **Dynamic memory allocation**
 - Know how much memory is needed after the program is run
 - Example: ask the user to enter from keyboard
 - Dynamically allocate only the amount of memory needed
- **C provides functions to dynamically allocate memory**
 - **malloc, calloc, realloc**

Memory Allocation Functions

- **malloc** : Allocates requested number of bytes and returns a pointer to the first byte of the allocated space
- **calloc** : Allocates space for an array of elements, initializes them to zero and then returns a pointer to the memory.
- **free** : Frees previously allocated space.
- **realloc** : Modifies the size of previously allocated space.

malloc

- A block of memory can be allocated using the function **malloc**
 - Reserves a block of memory of specified size and returns a pointer of type **void**
 - The return pointer can be type-casted to any pointer type

- General format:

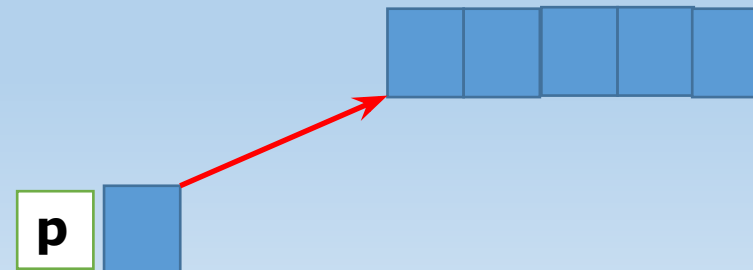
<data-type> *p;

p = (<data-type>*) malloc (byte_size);

```
p = (int *) malloc (5 * sizeof(int));
```

Reserves memory spaces for 5 integers.

p points to the address of the first integer



malloc

- **malloc** always allocates a block of contiguous bytes
 - The allocation can fail if sufficient contiguous memory space is not available
 - If it fails to allocate required memory, **malloc** returns **NULL**

Accessing Elements

```
int *p, n, i;  
scan("%d",&n);  
p=(int *) malloc(n*sizeof(int));
```

The n integers allocated by malloc can be accessed as

***p, *(p+1), *(p+2),..., *(p+n-1)**

or

p[0], p[1], p[2], ...,p[n-1]

Example

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *p,n,i;

    printf("Enter the Array Size:");
    scanf("%d",&n);

    printf("Array Elements are %d \n",n);

    p = (int*)malloc(n * sizeof(int));

    for (i = 0; i < n; ++i)
        p[i] = i + 1;

    printf("Array Elements: ");

    for (i = 0; i < n; ++i)
        printf("%d \n", *ptr+i);

    return 0;
}
```

Output:

```
Enter the array size: 6
Array Elements are 6
Array Elements:
1
2
3
4
5
6
```

free

- An allocated block can be returned to the system for future use by using the **free** function
- General syntax:
free (ptr);
where **ptr** is a pointer to a memory block which has been previously created using **malloc**
- No size needs to be mentioned for the allocated block, the system remembers it for each pointer returned

End of Lecture

References

- https://cse.iitkgp.ac.in/~bivasm/pds_notes/
- <https://www.geeksforgeeks.org/>
- <https://overiq.com/c-programming-101/>
- <https://www.oreilly.com/library/view/understanding-and-using/9781449344535/ch04.html>
- <https://www.javatpoint.com/pointer-arithmetic-in-c>
- https://www.cs.swarthmore.edu/~newhall/cs31/resources/C-structs_pointers.php
- <http://www2.ece.ohio-state.edu/~degroat/EG167C/Lecture%2014%20-%20Pointers%20-%2006.ppt>

End of Lecture